

NATIONAL ECONOMICS UNIVERSITY, HANOI
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE



Introduction to Databases

REPORT

Topic: Sports Ticketing Management System

Instructor: Dr. Tran Hung

Student: Pham Thuy Anh 11247137

Hanoi, May 2026

PROJECT 16: SPORTS TICKETING MANAGEMENT SYSTEM

DATCOM Lab
NEU-College of Technology
National Economics University
Email: hung.tran@neu.edu.vn

Project Objective

The objective of this project is to build a system to manage the sale of tickets for sports events, including scheduling, seat booking, customer management, and revenue reporting, ensuring fast, convenient, and accurate service for both organizers and spectators.

Database and Programming Languages

- **Database Management System:** MySQL
- **Programming Language:** Python

Detailed Project Requirements

System Analysis and Requirements

- Manage data related to sporting events, ticket types, customers, seating arrangements, ticket sales, and stadium box offices.

Main Functionalities

- Event scheduling and match information management.
- Ticket type and pricing management.
- Customer management and ticket purchasing history.
- Seat booking and ticket availability tracking.
- Ticket sales and box office management.
- Reporting on revenue, ticket sales, and event popularity.

Database Design and Implementation

1. Data Model Design

- Create an ER diagram showing the relationship among events, customers, tickets, and seating.
- Convert ER diagram to relational schema with PKs, FKs, and constraints.

2. Table Structures

- Events (EventID, EventName, EventDate, Venue)
- Tickets (TicketID, EventID, TicketType, Price)
- Customers (CustomerID, CustomerName, PhoneNumber, Address)
- BoxOffices (BoxOfficeID, OfficeName, Address)
- Seats (SeatID, EventID, SeatNumber, SeatType)

3. Sample Data

- Insert 510 rows for each table.
- Use MySQL Workbench to generate a schema diagram.

Advanced Database Objects

- **Indexes:** Speed up queries on available seats and customer purchases.
- **Views:** Generate views for sold-out events, revenue by event, or seat availability.
- **Stored Procedures:** Automate ticket booking and generate revenue summaries.
- **User Defined Functions:** Calculate total revenue per event or total tickets sold.
- **Triggers:** Update seat status when a ticket is sold or canceled.

Database Security and Administration

- Define user roles such as cashier, admin, and manager with appropriate access rights.
- Encrypt customer information and apply secure transaction handling.
- Set up backup plans and disaster recovery procedures.
- Optimize query performance for high-traffic sales periods.

Python Application Development

- **Database Connection:** Use `mysql-connector-python` or `SQLAlchemy`.
- **Booking System:** Implement Python scripts for searching events, booking tickets, and canceling orders.
- **Reporting Module:** Generate real-time sales statistics and revenue dashboards.
- **User Interface:** Provide a console-based or optional GUI for box office operations.

Deliverables

- A printed comprehensive report including database design, implementation details, and screenshots. Submit your report into LMS. First pages of your report must attach this PDF file.
- Link to publish Github source code and video presenting on Youtube
- SQL schema, sample data scripts, ER diagrams, and Python code.
- Demonstration of functionalities such as adding income, managing expenses, and generating reports.

Conclusion and Recommendations

- Summarize project success and performance.
- Recommend future enhancements such as QR code integration, online payment, or mobile ticketing apps.

References

- List all technical documents, libraries, and data sources used in the project.

Contents

1	Introduction	4
2	Introduction	4
2.1	Problem Definition	4
2.2	Project Objectives	4
2.3	Scope of the Project	5
3	Method & Experiment	6
3.1	System Requirements & Analysis	6
3.1.1	Functional Requirements	6
3.1.2	Non-Functional Requirements	6
3.2	Database Design	7
3.2.1	Relational Schema Overview	7
3.2.2	Table Relationships	8
3.2.3	Mapping: Logical Model to Physical Implementation	9
3.2.4	Detailed Table Descriptions	10
3.2.5	Detailed Table Logic & Design Rationale	15
3.3	Design Rationale & Business Logic	16
3.3.1	High-Volume Inventory Strategy	16
3.3.2	Dynamic Pricing Engine: Strategy & Implementation	16
3.3.3	Hybrid Security & Access Control	17
3.3.4	Seamless Identity Syncing (CRM Logic)	17
4	Implementation & System Results	17
4.1	Python Data Seeding Engine	17
4.1.1	ZoneConfiguration & Seat Generation	18
4.1.2	Seat Naming & Ticket Display Logic	18
4.1.3	Event Seat Population	19
4.2	Sample Data Overview	19
4.3	Database Indexes	20
4.4	Database Views	21
4.4.1	v_seat_availability	21
4.4.2	v_revenue_by_event	21
4.4.3	v_sold_out_events	22
4.4.4	v_customer_history	22

4.4.5	v_boxoffice_revenue	22
4.5	User-Defined Functions (UDFs)	23
4.5.1	fn_total_revenue(p_event_id)	23
4.5.2	fn_tickets_sold(p_event_id)	23
4.5.3	fn_sold_pct(p_event_id)	23
4.6	Database Triggers	24
4.6.1	trg_ticket_paid	24
4.6.2	trg_ticket_released	24
4.6.3	trg_check_soldout	24
4.7	Stored Procedures	25
4.7.1	sp_book_ticket	25
4.7.2	sp_confirm_payment	25
4.7.3	sp_cancel_ticket	25
4.7.4	sp_expire_locks	26
4.7.5	sp_revenue_summary	26
4.8	User Roles & Access Control	26
4.9	Backup & Disaster Recovery Plan	27
4.10	Query Optimization & Performance	27
5	Python Application	27
5.1	Database Connection	27
5.2	Booking System Logic	28
5.3	Reporting Module	30
5.4	User Interface	30
5.5	Multi-Seat Shopping Cart	31
6	Conclusion	32
6.1	Key Achievements	32
6.2	Limitations & Future Enhancements	32
	References	34



Source Code Information

The complete source code for this project is available at the following link:

[GitHub Repository: Project 16 - Sports Ticketing Management System](#)

1. Introduction

2. Introduction

2.1. Problem Definition

In the modern sports industry, the scale of events and the demand for seamless fan experiences have made traditional ticketing methods obsolete. Managing high-profile matches manually or through fragmented systems leads to several critical issues:

- **Data Integrity & Human Error:** Manual entries often result in duplicated bookings, incorrect seat assignments, or lost records, undermining the reliability of the entire system.
- **Inventory Management Challenges:** Without a real-time database, tracking available, locked, or sold-out seats across massive venues — such as the 40,341-capacity Stamford Bridge — becomes nearly impossible.
- **Delayed Financial Reporting:** Calculating revenue across various sales channels (Online, Agents, Venue) manually is time-consuming and prone to inaccuracies, preventing organizers from making data-driven decisions.
- **Concurrency Issues:** Handling thousands of simultaneous booking requests without a robust locking mechanism leads to race conditions where the same seat could be sold to multiple fans.

2.2. Project Objectives

The primary goal of this project is to develop a robust, automated Sports Ticketing Management System that leverages modern database technologies to streamline the entire event lifecycle. The specific objectives include:

- **Automated Scheduling & Inventory:** Building an integrated schema to manage stadium layouts, match schedules, and real-time seat availability automatically.
- **Transaction Integrity:** Implementing Stored Procedures and Database Triggers to ensure that every booking is atomic, consistent, and secure.
- **Multi-Channel Synchronization:** Consolidating sales data from Online platforms, third-party Agents, and physical Venue box offices into a single source of truth.

- **Enhanced Reporting:** Developing optimized Database Views and User-Defined Functions to provide instantaneous insights into revenue, attendance, and sold-out events.
- **Scalability & Performance:** Ensuring the system can handle high-volume data ingestion while maintaining sub-second query response times.

2.3. Scope of the Project

To ensure high-quality implementation and rigorous testing, the scope of this project is strategically defined as follows:

- **Geographic & Infrastructure Focus:** The system manages a network of 5 premium UK-based stadiums, including iconic venues such as Vitality Stadium, Stamford Bridge, and Craven Cottage.
- **Capacity & Data Volume:** The project simulates a total seating inventory of approximately 593,640 records to test database performance under real-world pressure.
- **Intelligent Seating Engine:** A core component is the development of an automated Python seeding engine that generates complex seating maps and event-specific seat records, ensuring data consistency across massive datasets.
- **Core Transactional Logic:** The system prioritizes the full order-to-ticket lifecycle, including atomic booking, dynamic pricing, and administrative reporting.
- **Multi-Seat Shopping Cart:** The booking engine supports multi-seat transactions, allowing customers to select and hold multiple seats across different zones in a single session.

3. Method & Experiment

3.1. System Requirements & Analysis

3.1.1. Functional Requirements

The system is designed to provide a comprehensive suite of features to automate the ticketing lifecycle:

- **Event & Infrastructure Management:** Automated creation and scheduling of sports events linked to specific stadium layouts.
- **Dynamic Ticket & Pricing Logic:** Support for multiple seat categories (VIP, Normal, Economy) with distinct base prices and automated pricing adjustments based on sales channel and demand.
- **Customer Relationship Management:** Maintenance of detailed customer profiles and historical purchase records to support both online and walk-in buyer flows.
- **Transaction Processing:** Real-time seat locking and secure payment confirmation through atomic database procedures.
- **Multi-Channel Sales:** Simultaneous support for Online, Agent, and Venue box office channels, with per-event channel authorization.
- **Advanced Reporting & Analytics:** Instantaneous generation of revenue summaries, attendance tracking, and sold-out event alerts via specialized Database Views.

3.1.2. Non-Functional Requirements

- **High-Performance Ingestion:** The system must process and seed tens of thousands of seat records without system failure or excessive latency.
- **Data Integrity & Security:** Strict referential integrity through foreign key constraints, and role-based access control for customer data protection.
- **Scalability:** The architecture must support an increasing volume of data as more stadiums and events are added.
- **Concurrency Control:** Reliable management of simultaneous user requests to prevent double-booking during high-demand sales windows.

3.2. Database Design

3.2.1. Relational Schema Overview

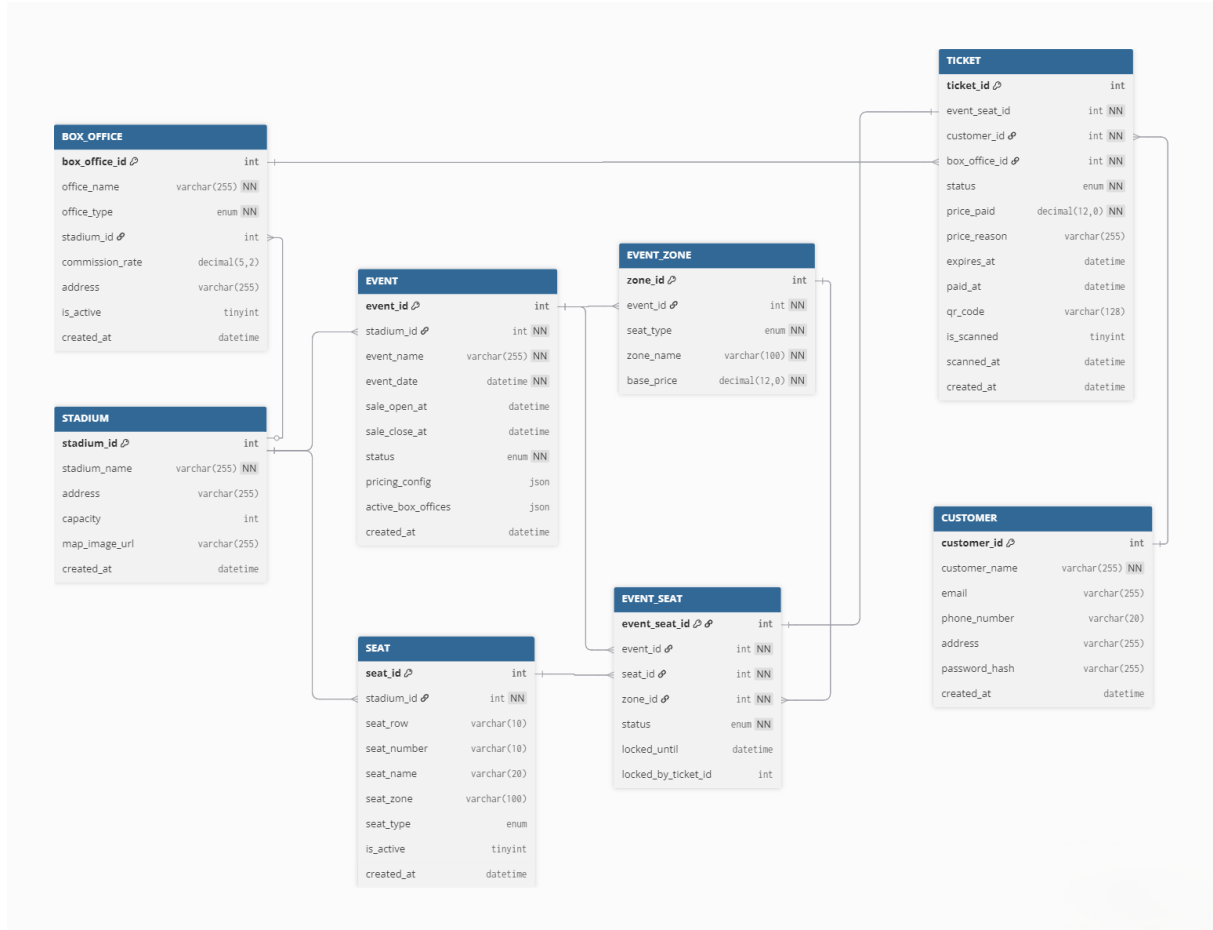


Figure 1: Relational Schema — Entity Relationship Diagram

The database is structured into 8 interconnected tables, optimized for Third Normal Form (3NF) to ensure data consistency and eliminate redundancy. The schema is organized into three logical layers:

- **Physical Infrastructure:** `STADIUM` and `SEAT` — static data that reflects the real-world venue layout.
- **Event Layer:** `EVENT`, `EVENT_ZONE`, and `EVENT_SEAT` — dynamic data that changes per match or show.
- **Transactions:** `CUSTOMER`, `BOX_OFFICE`, and `TICKET` — financial and identity records for every sale.

3.2.2. Table Relationships

Table 1: Entity Relationships Between Tables

Parent Table	Relation	Child Table	Join Key	Business Meaning
STADIUM	1 : N	EVENT	stadium_id	One stadium hosts many events
STADIUM	1 : N	SEAT	stadium_id	One stadium contains many physical seats
STADIUM	1 : N	BOX_OFFICE	stadium_id (nullable)	One stadium can have multiple venue counters
EVENT	1 : N	EVENT_ZONE	event_id	One event defines up to 3 pricing zones
EVENT	1 : N	EVENT_SEAT	event_id	One event tracks availability of all its seats
SEAT	1 : N	EVENT_SEAT	seat_id	One physical seat appears in many events
EVENT_ZONE	1 : N	EVENT_SEAT	zone_id	One zone groups many seats sharing the same price
EVENT_SEAT	1 : 1	TICKET	event_seat_id (UNIQUE)	One seat per event generates at most one ticket
CUSTOMER	1 : N	TICKET	customer_id	One customer can hold many tickets
BOX_OFFICE	1 : N	TICKET	box_office_id	One sales channel processes many transactions

3.2.3. Mapping: Logical Model to Physical Implementation

Table 2: Mapping of Logical Entities to Physical Tables

Original Entity	Physical Table(s)	Role & Transformation Logic
Events	EVENT	Stores match details, schedules, and stadium links.
Tickets	TICKET	Manages financial records, QR codes, and purchase statuses.
Customers	CUSTOMER	Stores unique identity and contact information for buyers.
Box Offices	BOX_OFFICE	Manages distribution channels and commission logic.
Seats	SEAT + EVENT_ZONE + EVENT_SEAT	Decomposed into three tables to handle static stadium layouts vs. dynamic event-specific availability.
Stadiums	STADIUM	The root entity providing infrastructure data (Capacity, Location).

3.2.4. Detailed Table Descriptions

The following subsections describe each table in the schema, including its purpose and a column-level breakdown of constraints and data types.

1. STADIUM — Physical Venues Manages core information about the physical sports venues. This is the root table; all seating and event data must reference a valid stadium record.

Table 3: **STADIUM** — Physical venues where events are held

Column Name	Constraints / Keys	Data Type & Notes
<code>stadium_id</code>	PK, AUTO_INCREMENT	INT NOT NULL
<code>stadium_name</code>	NOT NULL	VARCHAR(255)
<code>address</code>	—	VARCHAR(255)
<code>capacity</code>	—	INT UNSIGNED
<code>map_image_url</code>	—	VARCHAR(255)
<code>created_at</code>	NOT NULL, DEFAULT NOW()	DATETIME

2. EVENT — Matches & Shows Manages specific matches or scheduled sports events. It stores event timing, status, and the hosting stadium. Two JSON fields carry runtime configuration: `pricing_config` stores dynamic pricing rules, and `active_box_offices` defines which sales channels are authorized per event.

Table 4: **EVENT** — Events and matches; JSON fields drive dynamic behaviour

Column Name	Constraints / Keys	Data Type & Notes
event_id	PK, AUTO_INCREMENT	INT NOT NULL
stadium_id	FK → STADIUM(stadium_id), NOT NULL	INT
event_name	NOT NULL	VARCHAR(255)
event_date	NOT NULL	DATETIME
sale_open_at	—	DATETIME
sale_close_at	—	DATETIME
status	NOT NULL, DEFAULT 'Draft'	ENUM('Draft', 'OnSale', 'SoldOut', 'Cancelled', 'Finished')
pricing_config	—	JSON -- array of pricing rule objects
active_box_offices	—	JSON -- array of allowed box_office_id; NULL = all
created_at	NOT NULL, DEFAULT NOW()	DATETIME

3. SEAT — Static Seat Inventory Manages the permanent physical layout of every seat in each stadium. Seat type (VIP, Normal, Economy) is fixed by row position and configured during initial stadium setup. The `seat_name` is auto-generated by the Python seeding engine.

Table 5: **SEAT** — Physical seats; never deleted, only deactivated via `is_active`

Column Name	Constraints / Keys	Data Type & Notes
seat_id	PK, AUTO_INCREMENT	INT NOT NULL
stadium_id	FK → STADIUM(stadium_id), NOT NULL	INT
seat_row	—	VARCHAR(10) -- e.g. 'A', 'AA'
seat_number	—	VARCHAR(10) -- e.g. '01', '140'
seat_name	—	VARCHAR(20) -- e.g. 'MS1_A01'
seat_zone	—	VARCHAR(100) -- physical stand name
seat_type	—	ENUM('vip', 'normal', 'economy') NULL
is_active	NOT NULL, DEFAULT 1	TINYINT(1)
created_at	NOT NULL, DEFAULT NOW()	DATETIME

4. EVENT_ZONE — Per-Event Pricing Zones Sets ticket prices for specific seat categories during a particular event. Organizers can rename zones and redefine prices per event without altering the master seat data.

Table 6: **EVENT_ZONE** — One pricing row per seat type per event

Column Name	Constraints / Keys	Data Type & Notes
zone_id	PK, AUTO_INCREMENT	INT NOT NULL
event_id	FK → EVENT(event_id), NOT NULL	INT
seat_type	NOT NULL, UNIQUE with event_id	ENUM('vip', 'normal', 'economy')
zone_name	NOT NULL	VARCHAR(100) -- display name, may differ per event
base_price	NOT NULL	DECIMAL(10,2) -- base price in VND

5. EVENT_SEAT — Real-Time Inventory Tracker Tracks the real-time availability of every individual seat for every event. Contains over 482,000 records in the seeded dataset. Manages four states: Available, Locked, Booked, and Unavailable.

Table 7: **EVENT_SEAT** — Per-event seat inventory; links physical seats to zones

Column Name	Constraints / Keys	Data Type & Notes
event_seat_id	PK, AUTO_INCREMENT	INT NOT NULL
event_id	FK → EVENT(event_id), NOT NULL	INT
seat_id	FK → SEAT(seat_id), NOT NULL	INT
zone_id	FK → EVENT_ZONE(zone_id), NOT NULL	INT
status	NOT NULL, DEFAULT 'Available'	ENUM('Available', 'Locked', 'Booked', 'Unavailable')
locked_until	—	DATETIME -- expiry of Locked state
locked_by_ticket_id	Soft ref (no FK)	INT -- managed by application layer

6. CUSTOMER — Buyer Profiles Stores information for ticket buyers across all sales platforms. Row 1 is a pre-seeded anonymous sentinel record used for cash sales where no customer data is collected.

Table 8: **CUSTOMER** — Registered buyers; row 1 reserved for walk-in purchases

Column Name	Constraints / Keys	Data Type & Notes
customer_id	PK, AUTO_INCREMENT	INT NOT NULL
customer_name	NOT NULL	VARCHAR(255)
email	UNIQUE	VARCHAR(255) -- login credential for online users; NULL for walk-in
phone_number	UNIQUE	VARCHAR(20)
address	—	VARCHAR(255)
password_hash	—	VARCHAR(255) -- bcrypt hash; NULL for offline customers
created_at	NOT NULL, DEFAULT NOW()	DATETIME

7. BOX_OFFICE — Sales Channels Manages all sales points: official website, physical venue counters, and external agents. The `stadium_id` field links Venue-type counters to a specific stadium, while Online and Agent channels have this set to NULL.

Table 9: **BOX_OFFICE** — Sales channels: website, venue counters, external agents

Column Name	Constraints / Keys	Data Type & Notes
box_office_id	PK, AUTO_INCREMENT	INT NOT NULL
office_name	NOT NULL	VARCHAR(255)
office_type	NOT NULL	ENUM('Online', 'Venue', 'Agent')
stadium_id	FK → STADIUM(stadium_id), NULL	INT -- NULL for Online/Agent
commission_rate	NOT NULL, DEFAULT 0.00	DECIMAL(5,2) -- agent commission %
address	—	VARCHAR(255)
is_active	NOT NULL, DEFAULT 1	TINYINT(1)
created_at	NOT NULL, DEFAULT NOW()	DATETIME

8. TICKET — Financial Ledger The financial ledger for all successful sales. Records the actual price paid, payment timestamps, and a unique QR code identifier generated via `UUID()` for gate scanning.

Table 10: **TICKET** — Confirmed tickets; one per event_seat; UUID-based QR code

Column Name	Constraints / Keys	Data Type & Notes
<code>ticket_id</code>	PK, AUTO_INCREMENT	INT NOT NULL
<code>event_seat_id</code>	FK → EVENT_SEAT(event_seat_id), UNIQUE	INT NOT NULL
<code>customer_id</code>	FK → CUS- TOMER(customer_id), NOT NULL	INT
<code>box_office_id</code>	FK → BOX_OFFICE(box_office_id), NOT NULL	INT
<code>status</code>	NOT NULL, DEFAULT 'Pending'	ENUM('Pending', 'Paid', 'Cancelled', 'Expired')
<code>price_paid</code>	NOT NULL	DECIMAL(12,0) -- final price after pricing rules
<code>price_reason</code>	—	VARCHAR(255) -- e.g. 'Early Bird: -10%'
<code>expires_at</code>	—	DATETIME -- pending ticket expiry
<code>paid_at</code>	—	DATETIME
<code>qr_code</code>	UNIQUE	VARCHAR(128) -- opaque token for gate scanning
<code>is_scanned</code>	NOT NULL, DEFAULT 0	TINYINT(1)
<code>scanned_at</code>	—	DATETIME
<code>created_at</code>	NOT NULL, DEFAULT NOW()	DATETIME

3.2.5. Detailed Table Logic & Design Rationale

A. The Seating Infrastructure Logic ([SEAT](#), [EVENT_ZONE](#), [EVENT_SEAT](#)) The “Seat” entity is decomposed into three distinct tables to manage high-volume data efficiently and support flexible business requirements.

- **[SEAT](#) (Physical Infrastructure):** Stores the permanent physical location of every seat. Each seat is assigned a fixed type (VIP, Normal, Economy) based on its row position. A Unique Key on ([stadium_id](#), [seat_name](#)) prevents duplicate physical spots.
- **[EVENT_ZONE](#) (Pricing & Flexibility Engine):** Acts as the dynamic pricing layer for each event. Organizers can rename zones, set different base prices, or define only a subset of seat types per event.
- **[EVENT_SEAT](#) (Transaction & State Tracker):** The high-traffic table linking each physical seat to an event and its zone, while tracking real-time status through four states: Available, Locked, Booked, and Unavailable.

B. The Transactional Core ([TICKET](#))

- **Atomicity:** A ticket is only created if payment is confirmed and the corresponding [EVENT_SEAT](#) is successfully updated to “Booked”.
- **Fraud Prevention:** The `UUID()` generator provides a cryptographically unique, non-sequential identifier for every ticket, making it impossible to guess or replicate codes.
- **Performance:** A composite index on ([status](#), [expires_at](#)) optimizes the high-speed expiration logic within the 10-minute checkout window.

C. The Distribution Network ([BOX_OFFICE](#))

- **Global Channels:** When [stadium_id](#) is NULL, the entry represents a global distribution point (Official Website or third-party Agent).
- **On-site Venues:** When a specific [stadium_id](#) is assigned, the entry denotes a physical counter used only for events at that location.
- **Per-Event Authorization:** The [active_box_offices](#) JSON column in [EVENT](#) allows fine-grained control per event.

3.3. Design Rationale & Business Logic

3.3.1. High-Volume Inventory Strategy

The most significant architectural choice was decomposing the “Seat” entity into a three-tier structure: `SEAT`, `EVENT_ZONE`, and `EVENT_SEAT`.

The Problem: Physical seat locations are permanent, but their availability and pricing change for every single event. Storing all this in one table would lead to massive data redundancy and update anomalies.

The Solution: By separating static infrastructure from dynamic transaction states, the system can independently manage seat records for multiple concurrent events without performance degradation.

3.3.2. Dynamic Pricing Engine: Strategy & Implementation

The system utilizes a Dynamic Pricing Model processed via collaboration between the SQL database and the Python application layer. The `EVENT` table stores pricing rule parameters in a JSON column (`pricing_config`). Before a ticket is finalized, the Python engine evaluates these triggers and adjusts the base price.

The following rule types are supported:

- `days_before` — Time-based trigger: applies a modifier depending on how far in advance the purchase is made (e.g., Early Bird -10% if more than 30 days before the event).
- `sold_pct` — Demand-based trigger: applies a markup when a threshold percentage of seats has been sold (e.g., High Demand $+20\%$ when 85% of seats are booked).
- `velocity_1h` — Velocity-based trigger: applies a markup based on sales rate within a rolling 1-hour window.

Economic Rationale [5]

- **Inventory Perishability & Fixed Capacity:** Sports tickets are perishable inventory; their value drops to zero once the event concludes. Since stadiums have fixed seating capacity, the system must optimize revenue within a strictly limited supply.
- **Demand Fluctuations:** Ticket demand is volatile, influenced by team performance, match importance, and opponent prestige.

- **Market Segmentation:** The system leverages multiple pricing tiers (VIP vs. Normal vs. Economy) and channel-specific commission rates.

Model Simplification for Project Scope In production environments, dynamic pricing models are typically powered by Machine Learning models [1]. Due to the academic scope of this project, a simpler Rule-Based Pricing Configuration is implemented using conditional triggers (if-then logic) to demonstrate the core concept of real-time price adjustment in a transparent, auditable manner.

3.3.3. Hybrid Security & Access Control

- **UUID for Fraud Prevention:** Instead of sequential ticket IDs, the `UUID()` function generates a cryptographically unique identifier for each QR code, making ticket forgery effectively impossible.
- **JSON-Based Channel Authorization:** The `active_box_offices` JSON column in `EVENT` provides NoSQL-like flexibility within the relational schema, allowing instantaneous whitelisting of specific sales channels per event. For the current project scope, this approach is selected for its high efficiency. A dedicated junction table is noted as a future enhancement (see Section 6.2).

3.3.4. Seamless Identity Syncing (CRM Logic)

The `CUSTOMER` table supports a hybrid identity model. Walk-in customers can purchase tickets anonymously (routed to the sentinel record, `customer_id = 1`). If a customer who previously bought tickets at a venue later registers an online account using the same email, the system can automatically link their offline purchase history to their new digital profile, supporting long-term loyalty programs.

4. Implementation & System Results

4.1. Python Data Seeding Engine

A key component of this project is the automated Python seeding engine, implemented as a Jupyter Notebook. Rather than manually inserting seat data, the engine uses a declarative configuration system to generate and synchronize large volumes of seat records programmatically.

4.1.1. ZoneConfiguration & Seat Generation

The ZoneConfiguration dataclass is the core input object. Each instance defines a contiguous block of seats sharing the same type within a physical stand. Multiple configurations are stacked to represent different tiers within the same stand:

Listing 1: ZoneConfiguration example — Vitality Stadium

```
1 # Example: Vitality Stadium seat configuration
2 vitality_configs = [
3     # Main Stand: rows A-E = VIP, rows F-Z = Normal
4     ZoneConfiguration("Main Stand - VIP", "A", "E", 140, "vip"),
5     ZoneConfiguration("Main Stand - Normal", "F", "Z", 140, "normal"),
6
7     # North Stand: all Normal
8     ZoneConfiguration("North Stand - Normal", "A", "Z", 88, "normal"),
9 ]
10
11 # Sync all generated seat tuples to the database
12 sync_seats_to_db(connection, stadium_id=1, configs=vitality_configs)
```

The engine also supports multi-character row labels (AA, AB, ...) using an Excel-style encoding algorithm, removing any practical ceiling on stadium size.

4.1.2. Seat Naming & Ticket Display Logic

Each seat is assigned a structured `seat_name` that encodes its zone and position. The `get_zone_code()` function converts a full zone name into a compact code by extracting the initials of the stand name and appending a numeric type suffix:

Listing 2: Zone code encoding and seat_name generation

```
1 # Encoding logic
2 get_zone_code("Main Stand - VIP") # -> "MS1"
3 get_zone_code("North Stand - Normal") # -> "NS2"
4 get_zone_code("East Stand - Economy") # -> "ES3"
5
6 # Generated seat_name format: <ZoneCode>_<Row><Number>
7 seat_name = "MS1_A01" # Main Stand VIP, Row A, Seat 1
8 seat_name = "NS2_F07" # North Stand Normal, Row F, Seat 7
```

On a printed or digital ticket, the display format reads as:

Listing 3: Ticket label display format

```
1 # Ticket label format:
2 # <EventZoneName>: <ZoneCode>_<Row><Seat>
3
4 # Example outputs:
5 # "Platinum Hospitality: MS1_A01"
6 # "General Admission: NS2_F07"
```

This approach allows the event-facing zone name (stored in `EVENT_ZONE.zone_name`, e.g., “Platinum Hospitality”) to be displayed alongside the compact physical identifier (`SEAT.seat_name`, e.g., “MS1_A01”), giving attendees both context and precise location information.

4.1.3. Event Seat Population

Once physical seats and event zones are defined, the `EVENT_SEAT` table is populated via a single SQL query that joins the three layers together:

Listing 4: Event seat population query

```
1 INSERT IGNORE INTO EVENT_SEAT (event_id, seat_id, zone_id)
2 SELECT
3     e.event_id,
4     s.seat_id,
5     ez.zone_id
6 FROM EVENT e
7 JOIN SEAT s      ON s.stadium_id = e.stadium_id
8 JOIN EVENT_ZONE ez ON e.event_id  = ez.event_id
9                  AND s.seat_type  = ez.seat_type
10 WHERE s.is_active = 1
11 AND e.status      != 'Draft';
```

Draft events are excluded until the organizer publishes them. The query is idempotent — re-running it will not create duplicates due to the `INSERT IGNORE` and the unique key on (`event_id`, `seat_id`).

4.2. Sample Data Overview

To verify the system’s capability, a large-scale dataset was seeded including 5 stadiums and approximately 482,000 event seat records across 29 events.

	table_name	row_count
▶	STADIUM	5
	EVENT	29
	SEAT	106578
	EVENT_ZONE	77
	EVENT_SEAT	593640
	TICKET	482058

Figure 2: Table count

4.3. Database Indexes

Indexes are strategically implemented to optimize query performance, particularly for operations involving high-volume tables.

Listing 5: Index definitions

```
1 -- Speed up seat availability queries per event
2 ALTER TABLE event_seat ADD INDEX idx_eventseat_status (event_id, status);
3
4 -- Speed up ticket lookup by customer
5 ALTER TABLE ticket ADD INDEX idx_ticket_customer (customer_id);
6
7 -- Speed up event listing by status and date
8 ALTER TABLE event ADD INDEX idx_event_status_date (status, event_date);
9
10 -- Speed up expiry cron job queries
11 ALTER TABLE ticket ADD INDEX idx_ticket_status_expiry (status, expires_at)
12 ;
13 -- Speed up zone lookup per event
14 ALTER TABLE event_zone ADD INDEX idx_eventzone_event (event_id, seat_type)
15 ;
```

- **idx_eventseat_status (event_id, status):** During the booking phase, the most frequent query fetches available seats for a specific event. This composite index narrows the search space to only Available records for the selected `event_id`, dramatically reducing I/O on the largest table in the schema.
- **idx_ticket_customer (customer_id):** Optimizes purchase history lookups per customer, supporting the `v_customer_history` view and CRM features.
- **idx_event_status_date (status, event_date):** Accelerates queries that filter

events by their current status and date, used in the booking portal to display only OnSale events.

- **idx_ticket_status_expiry** (**status**, **expires_at**): Used by the `sp_expire_locks` cron procedure to efficiently locate expired Pending tickets without a full table scan.
- **idx_eventzone_event** (**event_id**, **seat_type**): Speeds up the join between `EVENT_ZONE` and `SEAT` during event seat population and pricing lookups.

4.4. Database Views

Views simplify complex data retrieval and provide a unified interface for reporting and ticket issuance.

4.4.1. `v_seat_availability`

Provides a real-time availability breakdown per zone per event, including counts of Available, Booked, Locked, and Unavailable seats, and a sold percentage. Used by the booking portal to render the seat map.

Listing 6: Querying seat availability

```
1 SELECT * FROM v_seat_availability WHERE event_id = 1;
2 -- Returns: zone_name, seat_type, base_price,
3 --           total_seats, available, booked, locked, sold_pct
```

	event_id	event_name	stadium_name	zone_name	seat_type	base_price	total_seats	available	booked	locked	unavailable	sold_pct
▶	1	Bournemouth vs Arsenal	Vitality Stadium	Premium	vip	60.00	2180	2180	0	0	0	0.0
	1	Bournemouth vs Arsenal	Vitality Stadium	Standard	normal	28.00	9156	9156	0	0	0	0.0

4.4.2. `v_revenue_by_event`

Aggregates total revenue, ticket count, and average price per event. Used in manager dashboards for financial reporting.

Listing 7: Querying revenue by event

```
1 SELECT * FROM v_revenue_by_event ORDER BY event_date DESC;
2 -- Returns: event_name, stadium_name, tickets_sold,
3 --           total_revenue, avg_price
```

	event_id	event_name	event_date	stadium_name	tickets_sold	total_revenue	avg_price
▶	1	Bournemouth vs Arsenal	2025-08-16 15:00:00	Vitality Stadium	10202	359771	35.2648
	2	Bournemouth vs Liverpool	2025-09-20 17:30:00	Vitality Stadium	10202	335843	32.9193
	3	Bournemouth vs Wolves	2025-11-01 15:00:00	Vitality Stadium	10202	226874	22.2382
	4	Bournemouth vs Man City	2025-12-26 17:30:00	Vitality Stadium	5793	227042	39.1925
	5	Bournemouth vs Chelsea	2026-02-14 15:00:00	Vitality Stadium	5759	173904	30.1969
	6	Luton vs Stoke City	2025-08-09 15:00:00	Kenilworth Road	10697	662001	61.8866
	7	Luton vs Sheffield Wed	2025-10-04 15:00:00	Kenilworth Road	10697	154671	14.4593
	8	Luton vs Burnley	2025-11-29 15:00:00	Kenilworth Road	6086	86862	14.2724
	9	Luton vs Derby County	2026-01-17 15:00:00	Kenilworth Road	5989	127852	21.3478
	10	Brentford vs Tottenham	2025-08-23 17:30:00	Gtech Community Stadium	13190	265378	20.1196

4.4.3. v_sold_out_events

Identifies events where every seat is booked. Used to automatically flag events for status update and halt further booking attempts.

Listing 8: Querying sold-out events

```
1 SELECT * FROM v_sold_out_events;
2 -- Returns: event_name, stadium_name, total_seats, booked_seats
```

4.4.4. v_customer_history

Consolidates the full purchase history per customer across all events and stadiums. Provides a ready-to-display format for customer account pages and CRM lookups.

Listing 9: Querying customer purchase history

```
1 SELECT * FROM v_customer_history WHERE customer_id = 2;
2 -- Returns: event_name, stadium_name, zone_name, seat_name,
3 --           price_paid, ticket_status, purchased_at, qr_code
```

	customer_id	customer_name	email	event_name	event_date	stadium_name	zone_name	seat_name	price_paid	price_reason	ticket_status	purchased_at
▶	2	James Wilson	james.wilson@outlook.com	Bournemouth vs Chelsea	2026-02-14 15:00:00	Vitality Stadium	Standard	SS2_Y54	24	Base Price	Paid	2026-05-05 00:23:07
	2	James Wilson	james.wilson@outlook.com	Bournemouth vs Chelsea	2026-02-14 15:00:00	Vitality Stadium	Standard	MS2_J100	24	Base Price	Pending	2026-05-05 00:23:07
	2	James Wilson	james.wilson@outlook.com	Bournemouth vs Chelsea	2026-02-14 15:00:00	Vitality Stadium	Standard	MS2_S129	24	Base Price	Pending	2026-05-05 00:23:07
	2	James Wilson	james.wilson@outlook.com	Bournemouth vs Chelsea	2026-02-14 15:00:00	Vitality Stadium	Standard	SS2_X12	24	Base Price	Paid	2026-05-05 00:23:07
	2	James Wilson	james.wilson@outlook.com	Bournemouth vs Chelsea	2026-02-14 15:00:00	Vitality Stadium	Standard	MS2_S088	24	Base Price	Paid	2026-05-05 00:23:07
	2	James Wilson	james.wilson@outlook.com	Bournemouth vs Chelsea	2026-02-14 15:00:00	Vitality Stadium	Premium	MS1_B028	52	Base Price	Paid	2026-05-05 00:23:07
	2	James Wilson	james.wilson@outlook.com	Bournemouth vs Chelsea	2026-02-14 15:00:00	Vitality Stadium	Standard	ES2_P039	28	Last Minute: +15%	Paid	2026-05-05 00:23:07

4.4.5. v_boxoffice_revenue

Summarizes tickets sold and total revenue per sales channel. Used to calculate agent commissions and compare channel performance.

Listing 10: Querying box office revenue

```
1 SELECT * FROM v_boxoffice_revenue ORDER BY total_revenue DESC;
2 -- Returns: office_name, office_type, tickets_sold, total_revenue
```

	box_office_id	office_name	office_type	tickets_sold	total_revenue
▶	12	Official Website	Online	186275	9493037
	14	StubHub UK	Agent	40863	2077780
	16	See Tickets	Agent	40329	2059627
	15	Viagogo	Agent	40719	2059428
	13	Ticketmaster UK	Agent	40342	2057187
	21	Stamford Bridge West Stand	Venue	19511	1001607
	18	Vitality East Gate Booth	Venue	19430	991727

4.5. User-Defined Functions (UDFs)

Functions handle modular calculations frequently required in financial reports.

4.5.1. fn_total_revenue(p_event_id)

Returns the total paid revenue for a single event by summing the final prices of all Paid tickets.

Listing 11: Using fn_total_revenue

```
1 SELECT fn_total_revenue(4) AS revenue;
2 -- Returns: 48500000 (total VND collected for event 4)
```

4.5.2. fn_tickets_sold(p_event_id)

Returns the count of Paid tickets for a single event.

Listing 12: Using fn_tickets_sold

```
1 SELECT fn_tickets_sold(4) AS sold;
2 -- Returns: 32410
```

4.5.3. fn_sold_pct(p_event_id)

Returns the percentage of seats booked for a single event. Used by the pricing engine to evaluate High Demand triggers.

Listing 13: Using fn_sold_pct

```
1 SELECT fn_sold_pct(4) AS pct;
2 -- Returns: 80.25 (80.25% of seats are booked)
```

4.6. Database Triggers

Triggers maintain real-time data consistency and automate status transitions without requiring additional application logic.

4.6.1. `trg_ticket_paid`

Fires AFTER UPDATE on `TICKET`. When a ticket's status transitions to "Paid", this trigger automatically updates the corresponding `EVENT_SEAT` from "Locked" to "Booked", clearing the lock timestamps.

Listing 14: Trigger: `trg_ticket_paid`

```
1 -- Trigger fires automatically when:
2 UPDATE ticket SET status = 'Paid', paid_at = NOW()
3 WHERE ticket_id = 101;
4 -- -> EVENT_SEAT.status becomes 'Booked'
5 -- -> locked_until and locked_by_ticket_id are cleared
```

4.6.2. `trg_ticket_released`

Fires AFTER UPDATE on `TICKET`. When a ticket is Cancelled or Expired, this trigger releases the associated seat back to "Available".

Listing 15: Trigger: `trg_ticket_released`

```
1 -- Trigger fires automatically when:
2 UPDATE ticket SET status = 'Cancelled' WHERE ticket_id = 101;
3 -- -> EVENT_SEAT.status reverts to 'Available'
```

4.6.3. `trg_check_soldout`

Fires AFTER UPDATE on `EVENT_SEAT`. When a seat transitions to "Booked", this trigger checks if any Available seats remain. If none are found, it automatically updates the `EVENT` status to "SoldOut".

Listing 16: Trigger: `trg_check_soldout`

```
1 -- Trigger fires automatically after every seat booking.
2 -- If no Available seats remain:
3 UPDATE event SET status = 'SoldOut' WHERE event_id = X;
```

4.7. Stored Procedures

Core business processes are encapsulated within Stored Procedures to ensure transaction atomicity and reduce network round-trips.

4.7.1. **sp_book_ticket**

Manages the entire booking transaction in a single atomic operation: validates seat availability, locks the seat, and creates a Pending ticket record. Uses `SELECT FOR UPDATE` to prevent concurrent bookings of the same seat.

Listing 17: Calling `sp_book_ticket`

```
1 CALL sp_book_ticket(  
2     1,          -- p_event_id  
3     42,         -- p_seat_id  
4     3,          -- p_customer_id  
5     1,          -- p_box_office_id (Online)  
6     500000,     -- p_price (calculated by Python pricing engine)  
7     'Early Bird: -10%', -- p_price_reason  
8     @ticket_id, @message -- OUT parameters  
9 );  
10 SELECT @ticket_id, @message;  
11 -- Returns: 1024, 'OK'  
12 -- OR:      NULL, 'UNAVAILABLE: Seat already taken'
```

4.7.2. **sp_confirm_payment**

Called by Python after the payment gateway confirms a successful transaction. Updates the ticket status from Pending to Paid, triggering `trg_ticket_paid` automatically.

Listing 18: Calling `sp_confirm_payment`

```
1 CALL sp_confirm_payment(1024, @msg);  
2 SELECT @msg; -- Returns: 'OK'
```

4.7.3. **sp_cancel_ticket**

Cancels a Pending or Paid ticket. Triggers `trg_ticket_released` automatically to release the seat back to Available.

Listing 19: Calling `sp_cancel_ticket`

```
1 CALL sp_cancel_ticket(1024, @msg);
2 SELECT @msg; -- Returns: 'OK'
```

4.7.4. sp_expire_locks

Run every 60 seconds by a Python cron job. Finds all Pending tickets whose `expires_at` has passed and releases their seats back to Available in a single batch UPDATE.

Listing 20: Calling `sp_expire_locks` (Python cron)

```
1 -- Called periodically by Python:
2 CALL sp_expire_locks();
3 -- Releases all seats locked by expired Pending tickets.
```

4.7.5. sp_revenue_summary

Generates a revenue report for all events at a given stadium, using the UDFs for aggregation. Used in the manager dashboard.

Listing 21: Calling `sp_revenue_summary`

```
1 CALL sp_revenue_summary(4); -- Stamford Bridge
2 -- Returns per-event: tickets_sold, total_revenue, sold_pct
```

4.8. User Roles & Access Control

To enforce the principle of least privilege, three database roles are defined:

Table 11: Database User Roles & Permissions

Role	Permissions	Use Case
<code>cashier</code>	SELECT on event/seat/customer tables; INSERT + UPDATE on ticket; EXECUTE on booking procedures	Venue staff selling and cancelling tickets at the box office
<code>manager</code>	SELECT on all tables and views; EXECUTE on reporting procedures and UDFs	Event organizers and administrators reviewing revenue and attendance
<code>admin_user</code>	ALL PRIVILEGES on <code>sports_ticketing_db</code>	Database administrators managing schema, users, and backups

4.9. Backup & Disaster Recovery Plan

Data integrity is maintained through a structured backup strategy. Automated backups are scheduled using the `mysqldump` utility with date-stamping:

Listing 22: Automated daily backup command

```
# Automated daily backup with date-stamping
mysqldump -u root -p sports_ticketing_db > backup_$(date +%Y%m%d).sql
```

This generates a complete logical backup including table structures, constraints, and all data. Backups are recommended on a daily schedule for active environments, with retention of at least 7 rolling snapshots.

4.10. Query Optimization & Performance

System responsiveness is prioritized through targeted indexing (see Section 3.3). The following additional strategies are applied:

- **Batch Insertions:** All Python seeding operations use `executemany()` rather than individual `INSERT` calls, reducing round-trips and leveraging MySQL's multi-row insert optimization.
- **SELECT FOR UPDATE:** Used within `sp_book_ticket` to serialize concurrent access to the same seat row, preventing race conditions without locking the entire table.
- **INSERT IGNORE with Idempotency:** The event seat population query uses `INSERT IGNORE`, making re-runs safe and avoiding duplicate key errors during partial failures.

5. Python Application

5.1. Database Connection

Database connection logic is isolated into a dedicated function. A connection pooling strategy is used to manage multiple concurrent requests efficiently.

Listing 23: Database connection setup

```
1 import mysql.connector
2
3 def connect_to_db():
4     connection = mysql.connector.connect(
5         host="127.0.0.1",
6         user="root",
7         password="<password>",
8         database="sports_ticketing_db"
9     )
10    return connection
```

5.2. Booking System Logic

The booking module acts as the bridge between the user interface and the `sp_book_ticket` stored procedure. The Python pricing engine evaluates dynamic pricing rules before calling the procedure:

Listing 24: Dynamic pricing engine

```
1 def calc_dynamic_price(base_price, event_date, sold_pct):
2     days_left = (event_date - datetime.now()).days
3
4     if days_left > 30:
5         return base_price * 0.90, "Early Bird: -10%"
6     elif days_left <= 3:
7         return base_price * 1.15, "Last Minute: +15%"
8     elif sold_pct >= 85:
9         return base_price * 1.20, "High Demand: +20%"
10
11    return base_price, "Base Price"
```


[← Back to Events](#)

CHELSEA VS MANCHESTER UNITED

30 Aug 2025 · 17:30 BST · Stamford Bridge, London

STAMFORD BRIDGE

Select a zone to book — hover to preview availability

Select Seats

Click a zone on the map to begin

Seats held for 5:33

West Stand VIP
 8,420 seats available

\$200

Number of tickets

−

1

+

Base price

\$200

Dynamic adj.

+20% (High Demand)

Service fee

\$10

Total

\$250

Confirm & Pay

Secure payment · Instant QR ticket delivery

Figure 3: Booking Interface

5.3. Reporting Module

The reporting module fetches aggregated data using the database views and UDFs, providing organizers with a clear overview of financial performance:

Listing 25: Reporting module queries

```
1 cursor.execute("SELECT * FROM v_revenue_by_event ORDER BY event_date DESC")
2 cursor.execute(
3     "SELECT * FROM v_seat_availability WHERE event_id = %s",
4     (event_id,)
5 )
6 cursor.execute("CALL sp_revenue_summary(%s)", (stadium_id,))
```

5.4. User Interface

The frontend web interface provides two distinct portals: a public ticket purchase portal for customers, and an administrative portal for event hosts and box office staff. The administrative portal is accessible only to accounts provisioned by the database administrator, aligning with the MySQL role-based access control defined in Section 3.8.

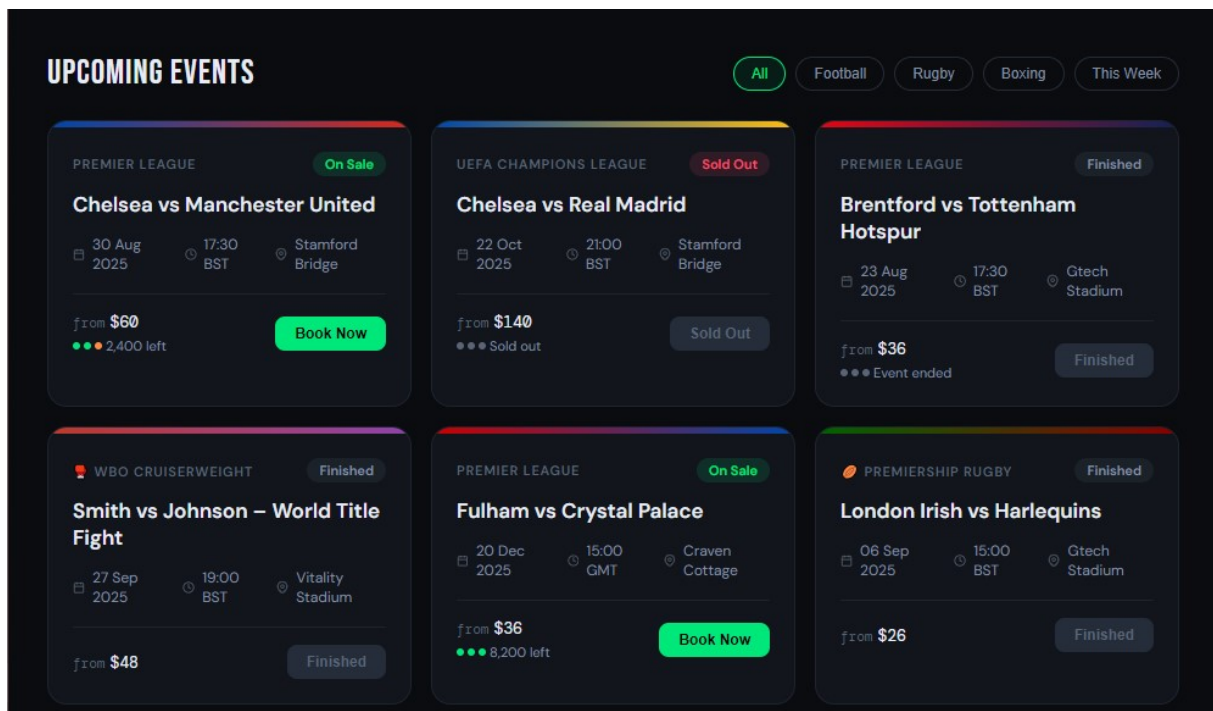


Figure 4: Customer Interface

PORTAL CAPABILITIES

Event Management
Create, edit, and cancel events. Set ticket tiers, pricing rules, and capacity limits per zone. Manage dynamic pricing multipliers.
[Open Event Manager →](#)

Booking Oversight
View all reservations, issue refunds, transfer bookings between customers, and override seat allocations for accessibility needs.
[View Bookings →](#)

Revenue Reports
Access real-time sales dashboards, export CSV/PDF reports, and view per-event P&L breakdowns with channel attribution.
[Open Reports →](#)

Access Control
View provisioned staff accounts and their assigned MySQL roles. Account creation is restricted to the database administrator only.
[View Access Log →](#)

ROLE-BASED ACCESS

MySQL Role Permissions Matrix					
ROLE	MANAGE EVENTS	VIEW BOOKINGS	ISSUE REFUNDS	REVENUE REPORTS	USER PROVISIONING
db_admin	✓ Full	✓ Full	✓ Full	✓ Full	✓ Full
event_host	✓ Own events	✓ Own events	— No	✓ Own events	— No
box_office	— No	✓ All	✓ Limited	— No	— No
readonly_staff	— No	✓ View only	— No	✓ View only	— No

STAFF SIGN IN

Administrative Access
Enter credentials provisioned by your database administrator to continue.

Staff Username
e.g. jsmith_host

Password

Role
box_office

[Sign In to Admin Portal](#)

Access restricted. This portal is available exclusively to staff accounts provisioned by the database administrator. Unauthorized access attempts are logged. Contact your DBA to request credentials.

Figure 5: Admin - Box office Portal

5.5. Multi-Seat Shopping Cart

The booking engine supports multi-seat transactions, allowing customers to select and hold multiple seats across different zones in a single checkout session. When a customer adds seats to their cart, each seat is immediately locked in `EVENT_SEAT` with a shared 10-minute expiry window. The Python layer tracks the basket state in memory and calls `sp_book_ticket` once per seat. On payment confirmation, `sp_confirm_payment` is called sequentially for each ticket in the basket. If payment fails or the session expires, `sp_expire_locks` releases all held seats atomically.

6. Conclusion

6.1. Key Achievements

Through the development of the Sports Ticketing Management System, several technical and structural objectives were successfully achieved:

- **Scalable Database Architecture:** A robust 8-table relational schema was designed and implemented, capable of managing high-volume data exceeding 482,000 records without performance degradation.
- **Optimized Inventory Management:** By decomposing the “Seat” entity into static ([SEAT](#)) and dynamic ([EVENT_SEAT](#)) components, the system efficiently handles real-time availability while maintaining a permanent infrastructure record.
- **Intelligent Seating Engine:** The Python `ZoneConfiguration` engine automates stadium layout definition and seat generation, supporting unlimited row depth via Excel-style multi-character row encoding.
- **Dynamic Pricing Logic:** A flexible pricing engine was established, utilizing JSON-based configurations and Python logic to adapt to time-based and demand-based triggers.
- **Transactional Integrity:** Stored Procedures and Triggers ensure that seat locking and ticket issuance are atomic, preventing double-booking and maintaining data consistency automatically.
- **Security & Optimization:** Data security is enhanced through UUID-based QR codes and role-based access control, while system responsiveness is ensured via strategic composite indexing.

6.2. Limitations & Future Enhancements

While the current system establishes a robust foundation, several advanced features are identified for future development:

- **Integrated Online Payment Gateways:** Integration with third-party payment APIs such as VNPay or Stripe will enable real-time payment confirmation and replace the current manual status update flow.

- **Mobile Application Development:** A dedicated mobile application is planned to support Digital Ticket Wallets and real-time notifications.
- **High-Concurrency Scalability:** Redis caching is planned to reduce database load during peak demand. A Virtual Queue system would manage high-volume requests fairly during flash sale scenarios.
- **ML-Based Dynamic Pricing:** The current rule-based pricing engine is intended to be replaced by Machine Learning models that analyze historical attendance, team performance metrics, and external factors.
- **Junction Table for Channel Authorization:** As the system scales, the JSON-based `active_box_offices` column would be migrated to a dedicated `EVENT_BOX_OFFICE` junction table to support referential integrity, advanced auditing, and channel-specific commission windows.

References

- [1] Alshamsi, M. *Smart Ticketing System using Blockchain*. MIT Thesis, 2022. Available at: <https://dspace.mit.edu/handle/1721.1/144487>
- [2] Lee, S. and Walsh, P. *The Impact of Technology on the Sports Ticketing Industry*. Journal of Purdue Undergraduate Research, 2015. Available at: <https://docs.lib.purdue.edu/jpur/vol15/iss1/15/>
- [3] Yusof, M. M., et al. *e-Ticketing system for football events in Malaysia*. ResearchGate, 2014. Available at: <https://www.researchgate.net/publication/261041862>
- [4] Buhalis, D. *The future of sports ticketing: Technologies, data and new strategies*. ResearchGate, 2023. Available at: <https://www.researchgate.net/publication/370331636>
- [5] Şahin, M. *Dynamic Pricing for Sports Events*. 1. Uluslararası Eğitim ve Sosyal Bilimlerde Yeni Ufuklar Kongresi (ICES-2018), pp. 482-487, Istanbul, TÜRKİYE, 2018. DOI: 10.21733/ibad.473973